

CSE 214 — Creational Design Patterns Exam Prep

Covers: **Singleton**, **Factory Method**, **Abstract Factory**, **Builder** Each section has: how to recognize the pattern in a question → general approach → reusable Java template → 5 practice questions modeled on your past papers.

1. SINGLETON PATTERN

How to recognize it in a question

Keywords: “single instance”, “shared”, “one and only one”, “in-memory config”, “must not have multiple copies”, “prevent second instance”, “same instance across the app”.

General approach

1. Make the constructor **private** so no outside class can call `new`.
2. Add a **private static** field of the class's own type to hold the one instance.
3. Add a **public static** `getInstance()` method that:
 - creates the instance the first time it's called (lazy), or
 - is created immediately at class load (eager).
4. Every client calls `ClassName.getInstance()` instead of `new ClassName()`.
5. To “prove” singleton behavior: get the instance from two places and show `instance1 == instance2` is true, or show a state change made via one reference is visible via the other.

General template

```
public class Singleton {
    // Step 1: single static instance, not created yet (lazy init)
    private static Singleton instance;

    // Step 2: private constructor blocks outside instantiation
    private Singleton() {
        // any one-time setup, e.g., loading a file
    }

    // Step 3: global access point
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    // example business method
    public void doSomething() {
        System.out.println("Using the single shared instance");
    }
}

// Client usage from two different places
class ClientA {
    void run() {
        Singleton s = Singleton.getInstance();
        s.doSomething();
    }
}

class ClientB {
    void run() {
        Singleton s = Singleton.getInstance();
        s.doSomething();
    }
}

public class Main {
    public static void main(String[] args) {
        new ClientA().run();
        new ClientB().run();
        System.out.println(Singleton.getInstance() == Singleton.getInstance()); // true
    }
}
```

Practice Questions

S1. You're building a `PrinterSpooler` for an office network. Multiple applications send print jobs, but there must be exactly one spooler managing the print queue so jobs aren't processed out of order or duplicated. Implement the `PrinterSpooler` class so all clients share the same instance, and demonstrate this from two different classes.

S2. A university's `DatabaseConnectionManager` should open only one connection object to the database and reuse it across the entire application to avoid the overhead of repeated connections. Implement this manager as a Singleton and show that two different modules (`StudentModule` and `CourseModule`) use the same connection instance.

S3. You are designing a `WeatherCache` for a mobile app. Fetching live weather data is expensive, so the app should fetch it once and store it in memory. All screens (Home, Forecast, Alerts) that need weather data must read from the same cached object. Implement `WeatherCache` as a Singleton and show that instantiating it from different screens always returns the same object.

S4. An e-commerce platform needs a `CartManager` that tracks the items a user has added, no matter which page (Home, ProductPage, Checkout) triggers cart access. There should never be two separate carts existing for the same user session in memory. Implement `CartManager` as a Singleton, and demonstrate adding an item from one class and reading it from another to confirm they share state.

S5. You are implementing a `ThemeManager` for a desktop application that stores the currently selected UI theme (e.g., "Dark" or "Light"). Every window/component in the app must read the same theme setting, and creating multiple `ThemeManager` objects would cause inconsistent UI across windows. Implement `ThemeManager` as a Singleton and demonstrate that two different UI components access the exact same instance.

2. FACTORY METHOD PATTERN

How to recognize it in a question

Keywords: "based on input type/string decide which class to instantiate", "client shouldn't know the concrete class name, only the interface", "add a new type later with minimal changes", "system creates and returns the appropriate object". Signal: **one product family**, created from **one factory/creator**, chosen by a **type string/enum**.

General approach

1. Define a common **interface** (or abstract class) for all products, with the shared method(s).
2. Create a **concrete class per product type**, each implementing the interface.
3. Create a **Factory class** with a method (e.g., `createX(String type)`) that:
 - takes the type indicator (string/enum) as a parameter,
 - uses `if/else` or `switch` to decide which concrete class to instantiate,
 - returns the object **typed as the interface**, not the concrete class.
4. The **client only talks to the interface and the factory** — it never writes `new ConcreteClass()` itself.
5. To extend later (new type), you only add a new concrete class + one new branch in the factory — client code is untouched.

General template

```
// Step 1: common interface
public interface Product {
    void action();
}

// Step 2: concrete products
public class ConcreteProductA implements Product {
    public void action() {
        System.out.println("Doing action as Product A");
    }
}

public class ConcreteProductB implements Product {
    public void action() {
        System.out.println("Doing action as Product B");
    }
}

// Step 3: factory that decides which class to instantiate
public class ProductFactory {
    public static Product createProduct(String type) {
        switch (type) {
            case "A":
                return new ConcreteProductA();
            case "B":
                return new ConcreteProductB();
            default:
                throw new IllegalArgumentException("Unknown type: " + type);
        }
    }
}

// Step 4: client only knows the interface + factory
public class Client {
    public static void main(String[] args) {
        Product p = ProductFactory.createProduct("A");
        p.action();
    }
}
```

Practice Questions

F1. You are building a `PaymentProcessor` module for an e-commerce site supporting `CreditCard`, `PayPal`, and `Crypto` payments. All payment types implement a common `PaymentMethod` interface with a method `pay()`. The client provides the payment type as a string, and the system should instantiate and return the correct payment object without the client knowing the concrete class names. Design the system so a new payment type (e.g., `"BankTransfer"`) can be added later with minimal changes.

F2. A document editor supports exporting files as `PDF`, `Word`, and `HTML`. All exporters implement a common `Exporter` interface with a method `export()`. Based on a string like `"PDF"` provided by the user, the system must create and return the correct exporter object. Implement this so the client code that triggers export never changes when a new export format is added.

F3. A ride-sharing app needs to create different types of rides — `Economy`, `Premium`, and `Pool` — all implementing a `Ride` interface with method `startRide()`. The client specifies the ride type as a string when requesting a ride. Implement a factory that returns the correct `Ride` object so the booking client code doesn't need to know about the specific ride classes.

F4. A customer support system routes tickets to different handlers: `EmailSupport`, `ChatSupport`, and `PhoneSupport`, all implementing a `SupportChannel` interface with method `handleTicket()`. Given a channel name as input, implement a system that creates and returns the correct handler object, keeping the ticket-routing client code unaware of the concrete classes.

F5. A media player needs to play different file types: `MP3Player`, `MP4Player`, and `WAVPlayer`, all implementing a `MediaPlayer` interface with method `play()`. Based on the file extension passed in as a string, implement a system that instantiates and returns the correct player object, allowing new formats to be added later without modifying the client.

3. ABSTRACT FACTORY PATTERN

How to recognize it in a question

Keywords: multiple **related/parallel product families** that must be created **together and stay consistent** (e.g., a UI toolkit needing matching `Button+Checkbox` for `Windows` vs `Mac`; matching `Engine+Tires+Seats` for `SUV` vs `Sedan`). Look for **two or more product types**, each with **multiple variants that belong to the same “family”/“theme”**. This differs from plain Factory Method because you're producing **more than one kind of related object per family**, not just one product type.

General approach

1. For each **product category** (e.g., `Button`, `Checkbox`), define a common **interface**.
2. For each **family/variant** (e.g., `Windows`, `Mac`), create **concrete classes for every product category** in that family.
3. Define an **AbstractFactory interface** declaring one creation method per product category (e.g., `createButton()`, `createCheckbox()`).
4. Create a **concrete factory per family** (e.g., `WindowsFactory`, `MacFactory`) implementing the abstract factory, returning the matching concrete products.
5. The **client** receives one concrete factory (based on some input) and uses only the abstract factory + product interfaces — guaranteeing all objects it builds belong to the same, consistent family.

General template

```

// Step 1: product interfaces (one per category)
public interface Button {
    void render();
}
public interface Checkbox {
    void render();
}

// Step 2: concrete products per family
public class WindowsButton implements Button {
    public void render() { System.out.println("Windows-style button"); }
}
public class WindowsCheckbox implements Checkbox {
    public void render() { System.out.println("Windows-style checkbox"); }
}
public class MacButton implements Button {
    public void render() { System.out.println("Mac-style button"); }
}
public class MacCheckbox implements Checkbox {
    public void render() { System.out.println("Mac-style checkbox"); }
}

// Step 3: abstract factory
public interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

// Step 4: concrete factories, one per family
public class WindowsFactory implements GUIFactory {
    public Button createButton() { return new WindowsButton(); }
    public Checkbox createCheckbox() { return new WindowsCheckbox(); }
}
public class MacFactory implements GUIFactory {
    public Button createButton() { return new MacButton(); }
    public Checkbox createCheckbox() { return new MacCheckbox(); }
}

// Step 5: client only depends on GUIFactory + interfaces
public class Application {
    private Button button;
    private Checkbox checkbox;

    public Application(GUIFactory factory) {
        button = factory.createButton();
        checkbox = factory.createCheckbox();
    }

    public void renderUI() {
        button.render();
        checkbox.render();
    }
}

public class Main {
    public static void main(String[] args) {
        GUIFactory factory = new WindowsFactory(); // or new MacFactory()
        Application app = new Application(factory);
        app.renderUI();
    }
}

```

```
}  
}
```

Practice Questions

AF1. You are building a cross-platform UI toolkit that must render consistent widgets for two operating systems: **Windows** and **Mac**. Each OS needs its own styled `Button` and `ScrollBar`, and a Windows `Button` must never be paired with a Mac `ScrollBar` in the same app. Implement a design where the client selects an OS and receives a matching, consistent set of widgets.

AF2. A car manufacturing simulator produces two vehicle families: **SUV** and **Sedan**. Each family requires a matching `Engine` and `Seats` type (e.g., SUV → “V6 Engine” + “Leather Seats”; Sedan → “4-Cylinder Engine” + “Fabric Seats”). Implement a system where selecting a vehicle family always returns correctly matched components — never mixing parts from different families.

AF3. A game engine must support two rendering families: **2D** and **3D**. Each family needs a matching `Renderer` and `SoundEngine` (e.g., 2D → “Sprite Renderer” + “Stereo Sound”; 3D → “Mesh Renderer” + “Surround Sound”). Implement a design ensuring a client that chooses the “3D” family always gets a 3D renderer paired with 3D-appropriate sound, never mismatched with 2D components.

AF4. A furniture store lets customers pick a design theme, **Modern** or **Victorian**, and each theme must supply a matching `Chair` and `Sofa`. Implement a system so that choosing a theme guarantees the chair and sofa produced always belong to that same theme, and adding a new theme (e.g., “Industrial”) requires minimal changes to client code.

AF5. A mobile OS abstraction layer needs to create matching `Notification` and `Dialog` UI components for **Android** and **iOS**. Each platform's notification style must always appear with that same platform's dialog style. Implement a design where selecting a platform returns a factory producing only that platform's consistent set of components.

4. BUILDER PATTERN

How to recognize it in a question

Keywords: “complex object with multiple parts/components”, “step-by-step construction”, “same construction process, different representations”, “director class”, “client only asks for a specific model/package and gets the fully assembled product”. Signal: **one object type**, but it has **several fields/components** that get set up piece by piece, and there are **preset “standard” configurations** (like Relaxation vs Adventure, Commuter vs Mountain Beast).

General approach

1. Identify the final **Product** class and its parts (fields), e.g., `HolidayPackage` with `flight`, `hotel`, `activity`.
2. Create a **Builder interface (or abstract class)** with one method per part (`setFlight()`, `setHotel()`, ...) plus a `getResult()` / `build()` method.

3. Create **concrete builders** — one per standard configuration (e.g., `RelaxationPackageBuilder` , `AdventurePackageBuilder`) — each hard-coding that configuration's values, OR a single generic builder with setter methods chained together.
4. Create a **Director class** that knows the *order* of construction steps and calls them on whichever builder it's given, then returns the finished product. This is what lets the client just say "build me a Relaxation package" without knowing the steps.
5. The **client** picks a builder, hands it to the director (or calls builder methods directly), and receives the completed product — same process, different resulting representations.

General template

```

// Step 1: final product
public class Product {
    private String partA;
    private String partB;
    private String partC;

    public void setPartA(String a) { this.partA = a; }
    public void setPartB(String b) { this.partB = b; }
    public void setPartC(String c) { this.partC = c; }

    public void show() {
        System.out.println(partA + " | " + partB + " | " + partC);
    }
}

// Step 2: builder interface
public interface Builder {
    void buildPartA();
    void buildPartB();
    void buildPartC();
    Product getResult();
}

// Step 3: concrete builders (one per "standard model")
public class ConcreteBuilderX implements Builder {
    private Product product = new Product();
    public void buildPartA() { product.setPartA("X-PartA"); }
    public void buildPartB() { product.setPartB("X-PartB"); }
    public void buildPartC() { product.setPartC("X-PartC"); }
    public Product getResult() { return product; }
}

public class ConcreteBuilderY implements Builder {
    private Product product = new Product();
    public void buildPartA() { product.setPartA("Y-PartA"); }
    public void buildPartB() { product.setPartB("Y-PartB"); }
    public void buildPartC() { product.setPartC("Y-PartC"); }
    public Product getResult() { return product; }
}

// Step 4: director – fixes the construction sequence
public class Director {
    public Product construct(Builder builder) {
        builder.buildPartA();
        builder.buildPartB();
        builder.buildPartC();
        return builder.getResult();
    }
}

// Step 5: client
public class Main {
    public static void main(String[] args) {
        Director director = new Director();

        Product p1 = director.construct(new ConcreteBuilderX());
        p1.show();

        Product p2 = director.construct(new ConcreteBuilderY());
        p2.show();
    }
}

```

```
}  
}
```

Practice Questions

B1. You are building a `MealPlan` system for a meal-prep company. A meal plan consists of a `MainCourse`, `SideDish`, and `Dessert`. The company offers two standard plans: **“Keto Plan”** (“Grilled Chicken”, “Steamed Broccoli”, “Sugar-Free Jello”) and **“Vegan Plan”** (“Tofu Stir-fry”, “Quinoa Salad”, “Fruit Bowl”). Implement a Builder-based design so the client only requests a plan name and receives the fully assembled `MealPlan`, using Strings for the components.

B2. A PC retailer lets customers order a `CustomPC` made up of a `CPU`, `RAM`, and `Storage`. Two standard configurations are offered: **“Gaming PC”** (“Intel i9”, “32GB RAM”, “1TB SSD”) and **“Office PC”** (“Intel i5”, “8GB RAM”, “512GB SSD”). Implement a system that separates the step-by-step construction of the `CustomPC` from its representation, so a client can request either standard build through the same construction process.

B3. An event management company builds `WeddingPackage` objects with a `Venue`, `Catering`, and `Decoration`. Two standard packages exist: **“Luxury Package”** (“Grand Ballroom”, “5-Course Meal”, “Floral Extravagance”) and **“Budget Package”** (“Community Hall”, “Buffet Meal”, “Simple Balloons”). Implement a Builder pattern so the construction steps stay the same regardless of which package is chosen, and the client just asks for a package by name.

B4. A computer game studio needs to construct `CharacterProfile` objects made of a `Weapon`, `Armor`, and `Skill`. Two preset character builds are offered: **“Warrior”** (“Broadsword”, “Plate Armor”, “Berserk Rage”) and **“Mage”** (“Staff of Fire”, “Cloth Robe”, “Fireball”). Implement a step-by-step Builder design (with a Director) so the client requests a build type and receives the fully assembled `CharacterProfile`.

B5. A furniture assembly company builds `PatioSet` objects consisting of a `Table`, `Chairs`, and `Umbrella`. Two standard sets are offered: **“Classic Set”** (“Wooden Table”, “4 Wooden Chairs”, “Beige Umbrella”) and **“Modern Set”** (“Glass Table”, “4 Metal Chairs”, “Black Umbrella”). Implement a Builder pattern (using Strings for components) that separates construction from representation, so the client can build either set through an identical construction process.

Quick Comparison Cheat-Sheet

Pattern	# of product types	# of variants/families	Key giveaway phrase
Singleton	1 object, 1 instance ever	—	“only one instance”, “same shared object everywhere”
Factory Method	1 product interface, many concrete classes	1 family	“decide which class to instantiate based on input string”, “client shouldn’t know concrete class”
Abstract Factory			

Pattern	# of product types	# of variants/families	Key giveaway phrase
	2+ related product interfaces	2+ families that must stay consistent	"matching set", "must not mix", multiple product categories per theme
Builder	1 complex object, many parts	2+ preset configurations	"step-by-step construction", "same process, different representations", "director"

Exam-day tips

- Read for **how many product types** and **how many families/variants** — that's the fastest way to tell Factory Method apart from Abstract Factory, and both apart from Builder.
- If the question only ever mentions **one object with no variation**, but stresses "only one copy in memory" → **Singleton**.
- If parts of an object are being **assembled step-by-step** and there are **named preset configurations** → **Builder**, and consider adding a `Director`.
- Always keep client code depending on **interfaces/abstract classes**, never concrete classes — examiners specifically check for this in every pattern above.
- Practice writing the **UML-style class relationships in words** (interface → concrete classes → creator/factory/builder → client) since you may be asked to explain your design, not just code it.

Good luck on your exam tomorrow!